

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 8 :

Classes et objets en Python : notions de bases

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Introduction
2. Toutes les variables de Python sont des classes
3. Syntaxe : créer une classe et instancier dans un objet
4. Héritage

Introduction

Motivation

Jusqu'à maintenant, nous avons vu différentes **fonctions prédéfinies** dans Python, telles que :

- print, type, len, sorted, sum, min, max, etc.

Et puis, nous avons vu comment on peut faire nos **propres fonctions** :

```
def somme(a, b):  
    res = a + b  
    return res
```

Aussi, nous avons vu différentes **modules et bibliothèques prédéfinies** dans Python, telles que :

- time, datetime, os, shutil, random, etc.

Et puis, nous avons vu comment on peut faire nos **propres modules et bibliothèques** :

- module : fichier Python contenant nos propres fonctions etc.

- bibliothèque : dossier contenant plusieurs modules

Et, nous avons vu différents **types de données** (int, float, str, etc.) et structures de données (list, dict, etc.)

Aujourd'hui, nous allons voir comment créer nos **propres classes de données**

Définitions

Classe :

Une classe est un modèle ou un plan pour créer des **objets**. Elle définit les caractéristiques communes à un groupe d'objets. Une classe peut contenir des attributs (variables) pour représenter l'état de l'objet et des méthodes (fonctions) pour définir le comportement de l'objet.

En termes plus concrets, une classe peut être comparée à un moule qui décrit la forme, la structure et les fonctionnalités d'un objet. Elle fournit un ensemble d'instructions sur la manière de créer un objet de cette classe, d'initialiser ses attributs et de lui permettre d'effectuer des actions spécifiques.

Par exemple, si nous avons une classe "Voiture", elle pourrait avoir des attributs tels que "marque", "modèle", "vitesse", et des méthodes telles que "démarrer", "accélérer", "freiner", etc.

Objet :

Un objet est une instance spécifique d'une classe. C'est une entité concrète qui est créée en utilisant le modèle défini par la classe. Chaque objet a ses propres valeurs pour les attributs de la classe et peut exécuter les méthodes définies par la classe.

Reprenons l'exemple de la classe "Voiture". Si nous créons un objet à partir de cette classe, par exemple une voiture nommée "voiture1", alors "voiture1" devient un objet de la classe "Voiture". Cet objet aura des valeurs spécifiques pour ses attributs tels que la marque, le modèle et l'année de fabrication, qui peuvent être différents d'un autre objet de la même classe.

Les objets sont utilisés pour représenter des entités du monde réel dans un programme informatique. Chaque objet est distinct et peut interagir avec d'autres objets ou exécuter des actions selon les méthodes définies dans sa classe.

En quelques sortes : c'est comme la définition de la fonction (def nom(arg1, arg2) etc.)
et la fonction lors de l'appel avec des valeurs concrètes pour les arguments

**Toutes les variables de Python
sont des **classes****

Variables primitives

```
a = 1  
b = 2.5  
c = True  
d = 'abc'  
e = 2 + 3j
```

```
print(type(a))  
print(type(b))  
print(type(c))  
print(type(d))  
print(type(e))
```

```
<class 'int'>  
<class 'float'>  
<class 'bool'>  
<class 'str'>  
<class 'complex'>
```


Structures de données

```
a = [1, 2.5, 3, 'mohamed']  
b = {1, 2, 3, 'mohamed', 2, 3}  
c = {1:'a', 2:'b', 3:'c', 4:'mohamed'}  
d = (1, 2, 3, 'mohamed')
```

```
print(type(a))  
print(type(b))  
print(type(c))  
print(type(d))
```

```
<class 'list'>  
<class 'set'>  
<class 'dict'>  
<class 'tuple'>
```

Classes parvenant des modules et bibliothèques

```
import datetime
```

```
a = datetime.datetime.now()  
print(a)
```

```
2024-03-20 10:16:17.620299
```

```
print(type(a))
```

```
<class 'datetime.datetime'>
```

```
import tkinter as tk
```

```
root = tk.Tk()  
print(type(root))
```

```
<class 'tkinter.Tk'>
```

```
import requests
```

```
res = requests.get('https://www.uca.ma/fssm/en')  
print(type(res))
```

```
<class 'requests.models.Response'>
```

```
import itertools
```

```
a = itertools.combinations([1, 2, 3, 4, 5], 2)  
print(type(a))
```

```
<class 'itertools.combinations'>
```

Syntaxe:

**Comment définir une classe et
créer des objets**

`__init__` et `self`

Syntaxe, __init__, self, attributs, méthodes

```
class utilisateur:
    def __init__(self, nom, age):  #__init__ pour initialiser les attributs
        self.nom = nom
        self.age = age

    def présenter(self):           # self pour avoir l'accès aux attributs depuis cette méthode
        print("bonjour, je suis : ", self.nom)
        print("j'ai : ", self.age, "ans")
```

__init__ aussi appelée
constructeur

```
## type :
u1 = utilisateur("Mohamed", 20)
print(type(u1))
```

```
<class '__main__.utilisateur'>
```

```
# attributs
print(u1.nom)
print(u1.age)
```

Mohamed
20

```
# méthode
u1.présenter()
```

bonjour, je suis : Mohamed
j'ai : 20 ans

Pourquoi self

```
class utilisateur:
    def __init__(self, nom, age):
        couleur = "couleur"
        self.nom = nom
        self.age = age

    def présenter(self):
        print("bonjour, je suis : ", self.nom)
        print("j'ai : ", self.age, "ans")

    def couleur(self):
        print(couleur)
```

```
mohamed = utilisateur("Mohamed", "20")
```

```
mohamed.présenter()
```

```
bonjour, je suis : Mohamed
j'ai : 20 ans
```

```
mohamed.couleur()
```

NameError

Traceback (most recent call last)

Cell In[77], line 1

----> 1 mohamed.couleur()

Cell In[67], line 12, in utilisateur.couleur(self)

```
    11 def couleur(self):
----> 12     print(couleur)
```

NameError: name 'couleur' is not defined

Héritage

```
class Employee:
    def __init__(self, name, salary, age, exp):
        self.name = name
        self.salary = salary
        self.age = age
        self.experience = exp

    def display_info(self):
        print(f"Name: {self.name}, Salary: {self.salary}, \
              age: {self.age}, experience: {self.experience}")
```

```
employee = Employee("Mohamed", 10000, 40, 10)
employee.display_info()
```

Name: Mohamed, Salary: 10000, age: 40, experience: 10

Classe mère

Dans cet exemple utilisant l'héritage, la classe `Manager` hérite de la classe `Employee`, ce qui signifie que la classe `Manager` obtient automatiquement les attributs et les méthodes de la classe `Employee`. Cela réduit la redondance du code, car nous n'avons pas besoin de redéfinir les attributs et les méthodes communs dans la classe `Manager`.

`super()` est une fonction intégrée en Python qui est utilisée pour accéder et appeler les méthodes et attributs de la classe parente (ou classe de base ou classe mère) dans une classe dérivée (ou classe fille) lors de l'utilisation de l'héritage. C'est surtout utile pour rajouter des attributs complémentaires sans avoir besoin de réinitialiser les attributs de la classe parente

Classe fille

```
## manager est un employé aussi, sauf qu'il a une information supplémentaire
## qui est le département qui gère
```

```
class Manager(Employee):
    def __init__(self, name, salary, age, exp, department):
        ## super fait référence à la classe mère
        super().__init__(name, salary, age, exp)
        self.department = department

    def display_info(self):
        super().display_info()
        print(f"Department: {self.department}")
```

```
manager = Manager("Yassine", 20000, 50, 15, "Production")
manager.display_info()
```

Name: Yassine, Salary: 20000, age: 50, experience: 15
Department: Production

Autre exemple

```
class Vehicle:
    def __init__(self, a, b, c, d, e, f):
        self.couleur = a
        self.pos = b
        self.vitesse = c
        self.nbr_portes = d
        self.nbr_roues = e
        self.taille = f

    def transporter(self):
        return self.pos + self.vitesse
```

```
V = Vehicle("bleu", 4, 80, 4, 4, 3)
V.transporter()
```

84

si on va pas modifier dans la fonction __init__, on peut ne pas utiliser super

```
class Bus(Vehicle):
    def transporter(self, stops):
        return self.pos + self.vitesse - stops

B = Bus("Noire", 4, 80, 4, 4, 3)
B.transporter(10)
```

74

ajouter un seul attribut

```
class Bus(Vehicle):
    def __init__(self, a, b, c, d, e, f, g):
        super().__init__(a, b, c, d, e, f)
        self.stops = g

    def transporter(self):
        return self.pos + self.vitesse - self.stops

B = Bus("jaune", 4, 80, 4, 4, 3, 10)
B.transporter()
```

74

ou modifier dans la méthode

```
class Bus(Vehicle):
    def __init__(self, a, b, c, d, e, f):
        super().__init__(a, b, c, d, e, f)

    def transporter(self, stops):
        return self.pos + self.vitesse - stops

B = Bus("jaune", 4, 80, 4, 4, 3)
B.transporter(10)
```

74